

Guest Lecture: Redesigning Transformers for Efficiency Algorithmic and System Perspectives

Lecturer: Han Guo Scribe: [Yupei Wang, Edith Gu, Sukruth Rao, Wenqi Li, Lenan Wu, Feiyang Jiang]

1 Introduction and Attention as a Sequence Modeling Primitive

The lecture began with Minshen Zhang introducing Han Guo. Han is a PhD student at MIT CSAIL, advised by Yoon Kim and Eric Xing, working on algorithms and systems for scalable and efficient machine learning. Before MIT, he completed his bachelor's degree in computer science and statistics at UNC Chapel Hill, worked with the UNC NLP group, and spent several years at CMU's Language Technology Institute. His recent work includes Log-Linear Attention, LQLoRA, and efficient LLM quantization.

1.1 Roadmap of the Lecture

Han framed the talk as two complementary approaches to making Transformers more efficient. The first half is algorithmic: can we replace the main sequence-modeling primitive with something that scales better asymptotically? This motivates linear attention and related recurrent or state-space-style architectures. The second half is systems-oriented: can we keep a Transformer-like architecture but rewrite its computations so they better match modern GPU kernels, especially GEMM and epilogue patterns?

1.2 The Standard Language-Modeling Recipe

Han next reviewed the standard recipe for training modern language models:

- **Data.** Models are trained on very large corpora, often at the scale of trillions of words or tokens.
- **Model.** The dominant architecture is the Transformer, which maps parameters θ to a probability model $p_\theta(\cdot)$ over text.
- **Optimization.** The usual objective is maximum likelihood, which can also be viewed as compression of the observed corpus.

In this picture, pre-training absorbs knowledge from the external world that generated the data. Post-training adapts the model using user preferences or reinforcement-learning-style procedures. At test time, search or test-time scaling can further improve answers. Han's point was that, even with this successful recipe, the architecture itself is still worth redesigning for efficiency.

1.3 Why Architecture Is the Focus

Architecture determines the basic computational shape of the model. Better data, larger models, and stronger optimization help, but if the core sequence-modeling primitive has an expensive scaling law, long-context use cases will eventually hit that cost. The first half of the lecture therefore focuses on attention, the main mechanism that lets tokens interact with other tokens in the same context.

1.4 Attention as a Sequence-Modeling Primitive

For language modeling, attention contextualizes the current word or token using the previous words in the sequence. The slide example was the phrase “UCSD is located in”. To predict the next word, the model should use the earlier words and infer that a plausible next word is “San”. More generally, each token representation can look back at other token representations in the context.

The key benefit is that attention models rich interactions among input elements. Instead of forcing all past information through a single running hidden state, attention lets one position depend directly on many other positions. This is why attention became the central primitive for accurate sequence modeling, and it is the behavior that efficient alternatives must try to preserve.

2 Attention Limitations

Before diving into its limitation, attention gained traction due to the following reasons:

- **Parallelizability:** The number of sequential operations in attention is independent of sequence length, hence, given sufficient parallel compute, the entire attention matrix can be computed without sequential dependencies across time steps. In practice, optimizations like FlashAttention make attention memory-linear during training, and at context lengths up to roughly 64K tokens, attention is generally not the training bottleneck.
- **Direct access / In-context retrieval:** Unlike RNNs, where past information must be recovered through a compressed hidden state, transformers let every token directly access any prior token’s representation, enabling strong arbitrary recall from context.

2.1 Long-Context Scalability

The first major limitation of standard attention is scalability at long sequence lengths. The attention matrix has size $L \times L$, so computing it scales as $O(L^2)$ in sequence length. While practical contexts today might be a few hundred tokens, real-world use cases demand much more. It could be entire books, full codebases (tens of millions of tokens), or even biological sequences at the DNA level. Plugging a standard Transformer into an attention algorithm struggles at these scales.

2.2 KV Cache Growth at Inference

During training, FlashAttention’s memory-efficient recomputation keeps memory usage linear. At inference, however, the situation is different. To avoid recomputing key and value projections for all previous tokens at each generation step, practitioners cache them (KV cache). This cache grows linearly with sequence length

and is never discarded during a single inference session. For long-context applications such as a coding agent reading an entire repository, the KV cache can easily exceed the memory footprint of the model weights themselves, which makes naive deployment of long-context models extremely memory-intensive.

2.3 Expressivity and State Tracking

The third limitation is more theoretical in nature. The guest speaker introduced the state tracking task: given a sequence of updates to a set of objects (for example, a series of boxes being moved between locations), can the model correctly report the final state of each object? Intuitively this requires maintaining and updating a consistent world state across time. It has been shown that Transformers without chain-of-thought are contained in the circuit complexity class TC^0 . The practical implication is that state tracking tasks require Transformers to use very deep layers to solve efficiently; a bounded-depth Transformer cannot handle them in the general case. Models in practice are often deep enough to approximate the task reasonably well, so the limitation matters more in theory than in everyday benchmarks. Nevertheless, it demonstrates that Transformer attention was not designed with this class of tasks in mind.

The three limitations motivate the central question of the talk: can we design a new sequence modeling primitive that preserves attention’s parallelism and direct-access benefits during training, while achieving constant-memory inference and stronger expressivity on tasks that standard attention handles poorly? The remainder of the talk addresses this question through the lens of linear attention.

3 Linear Attention Basics

In this part, linear attention is introduced as the main alternative primitive to softmax attention. The discussion proceeds in four steps: (1) linear attention is softmax attention with the softmax removed, which exposes an equivalent recurrent form with a matrix-valued state; (2) this gives constant-memory inference, but a naive version fails because the fixed-size state fills up, which motivates data-dependent forgetting gates; (3) because the state transition is linear, training can use a chunkwise algorithm that is both parallel and memory-efficient; and (4) richer, matrix-valued state transitions, DeltaNet in particular, can make these models more expressive, not just more efficient.

3.1 Dropping the softmax

The “modern” form of linear attention dates to around 2020 (Katharopoulos et al., 2020). Conceptually it is simple: take causal softmax attention,

$$o_t = \sum_{i=1}^t \frac{\exp(q_t^\top k_i)}{\sum_{j=1}^t \exp(q_t^\top k_j)} v_i,$$

and drop the exponential and the denominator:

$$o_t = \sum_{i=1}^t (q_t^\top k_i) v_i.$$

The equation looks almost identical, just without the nonlinearity. The natural question is why anyone should care.

3.2 The recurrent view: a linear RNN with a matrix-valued state

The answer is that without the softmax, the computation can be reordered. Instead of computing query-key scores first ($QK^\top$, then multiplying by V), we can accumulate the key-value outer products first and apply the query at the end:

$$o_t = \left(\sum_{i=1}^t v_i k_i^\top \right) q_t \iff S_t = S_{t-1} + v_t k_t^\top, \quad o_t = S_t q_t.$$

This is the same model, but with a *recurrent* schedule: maintain a running state S_t , update it with each new token, and unroll it for as long as the sequence goes. The result is essentially a **linear RNN**, with two differences from classical RNNs/LSTMs:

1. The state transition is a *linear* function (no nonlinearity between state updates; here, the update is just adding $v_t k_t^\top$).
2. The hidden state is *matrix-valued*: instead of a d -dimensional vector as in the RNN/LSTM days, the state is a $d \times d$ matrix. This much larger state is a big part of why modern linear attention works well in practice.

The efficiency consequences are linear compute and **constant memory at inference** (there is only one state to carry, no growing KV cache), which is what conceptually enables much longer sequences.

3.3 Why the naive version fails: forgetting gates

This sounds amazing, so can we just use it? Not quite. With a purely additive update, information keeps being *added* into a fixed-size state. The state is like a disk of fixed capacity: if you keep writing checkpoints to it, you eventually run out of space and must delete something before you can write again. This is the same problem that motivated LSTMs over vanilla RNNs.

The fix, from the data-dependent gating line of work (e.g., Gated Linear Attention), is to bring LSTM-style **data-dependent forgetting gates** back into linear attention:

$$S_t = G_t \odot S_{t-1} + v_t k_t^\top,$$

so the model can erase old information and free capacity for new information. This gated view is surprisingly general: depending on how the gate G_t is parameterized, it recovers a wide spectrum of today’s models. Mamba and Mamba-2 are, in this sense, particular gate parameterizations, and GLA and the later Gated DeltaNet can also be read as different gate designs. Gating is the key step that made linear attention practical, with quality approaching transformers.

3.4 Training efficiently: the chunkwise parallel algorithm

Memory efficiency is not enough; the reason the field moved from LSTMs to transformers was *training* efficiency. A classical recurrent implementation unrolls the state one step per token, which is a sequential loop that becomes too expensive at millions or billions of tokens, while transformers parallelize across the sequence.

Linear RNNs admit a middle ground. Because the state transition is linear, one can use a **chunkwise algorithm**: split the sequence into chunks; *within* each chunk, run a highly parallel (attention-like, matmul-heavy) computation; *across* chunks, run a recurrence that passes the chunk-level state forward, combining

the previous chunk’s state with the current chunk’s contributions to produce outputs. With a large enough chunk size, you get both GPU-friendly parallelism and a small memory footprint. (The detailed derivation is omitted here.)

Essentially all frontier linear-attention-style models in use today (Mamba-2, DeltaNet, and others) train with some flavor of the chunkwise algorithm. Different papers name it differently, but it is the same idea, and it is the main reason these models map so well onto GPUs.

3.5 Expressivity: state tracking and richer state transitions

The discussion then turns from efficiency to expressivity. Recall from earlier in the talk: transformers have theoretical limits on **state tracking** tasks. Linear attention out of the box is no better, and probably a little worse in practice, while *nonlinear* RNNs handle state tracking well but are not parallelizable.

The surprising point: **if the state-transition matrix is designed well, linear attention can in certain ways exceed transformers in expressivity**. An important caveat: the result is theoretical, in the sense that one can construct synthetic tasks where these models do very well, and at large-scale training it is hard to attribute quality to any single architectural feature, so the practical importance is debatable. Still, on certain tasks, the more efficient model can also be the more capable one.

The enabling change is a richer state transition. Earlier linear-attention variants (roughly the 2020–2023/24 generation) used an **elementwise (Hadamard) product**: a per-element gate that decays each entry of the state. More recent models (the Mamba-2/DeltaNet generation) move to **matrix-valued (“matmul”) state transitions**, where the state is updated by multiplying with a transition matrix M_t :

$$S_t = S_{t-1}M_t + v_t k_t^\top.$$

3.6 The DeltaNet setup

DeltaNet is the running example of a matmul-style transition. Its transition matrix is “identity minus a low-rank (rank-one) term”:

$$M_t = I - \beta_t k_t k_t^\top, \quad S_t = S_{t-1}(I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top.$$

The mechanism matters more than the exact algebra. At each step the model (1) looks at the last recurrent state and asks, for the current key k_t , what old value is currently stored, i.e., retrieves $v_t^{\text{old}} = S_{t-1}k_t$; (2) combines that old value with the newly received value v_t using a learnable scalar gate β_t , giving $v_t^{\text{new}} = (1 - \beta_t)v_t^{\text{old}} + \beta_t v_t$; and (3) writes the result back, replacing the old association. The output is then computed in the usual linear-attention way ($o_t = S_t q_t$, followed by output projections). Writing out this retrieve-combine-write procedure yields exactly the closed form above (we omit the derivation).

The point of the design: the model can **update and remove key-value associations**, not just accumulate or decay them, and this is the key to the added expressivity, in theory and to some extent in practice.

4 DeltaNet, and Associative Recall

4.1 DeltaNet as an Editable Memory

This portion of the lecture continues the discussion of linear attention by focusing on DeltaNet. The main idea is to make the recurrent state behave like an editable key-value memory: when the same key is encountered

again, the model should be able to remove or update the value associated with that key rather than simply adding another write to the state.

Using the convention shown on the DeltaNet slide, the key, query, and value vectors are linear projections of the current token representation:

$$q_t, k_t, v_t = W_Q x_t, W_K x_t, W_V x_t. \quad (1)$$

The state matrix S_t stores a compressed set of key-value associations. Before writing the current value, DeltaNet first retrieves the old value currently associated with the key:

$$v_t^{\text{old}} = S_{t-1} k_t. \quad (2)$$

It then forms an interpolated replacement value

$$v_t^{\text{new}} = \beta_t v_t + (1 - \beta_t) v_t^{\text{old}}, \quad (3)$$

where β_t controls how aggressively the current value overwrites the old memory. The state update can be written as

$$S_t = S_{t-1} - \underbrace{v_t^{\text{old}} k_t^\top}_{\text{remove old memory}} + \underbrace{v_t^{\text{new}} k_t^\top}_{\text{write new memory}} \quad (4)$$

$$= S_{t-1} + \beta_t (v_t - v_t^{\text{old}}) k_t^\top. \quad (5)$$

The output is then read from the updated state:

$$o_t = S_t q_t. \quad (6)$$

The speaker emphasized that this is an application of the delta update rule, an old idea going back to Widrow and Hoff (1960). In this setting, the rule gives a linear-attention state a more active editing mechanism than a pure additive write.

4.2 Structured Matrix Recurrences

The next slide generalized the scalar or elementwise gating view of linear transformers into a matrix-valued recurrence. A typical gated linear recurrence is of the form

$$S_t = G_t \odot S_{t-1} + k_t v_t^\top. \quad (7)$$

The structured matrix recurrence replaces the elementwise gate with a transition matrix:

$$S_t = M_t S_{t-1} + k_t v_t^\top. \quad (8)$$

This makes the transition more expressive, but it also raises the central systems question: how can M_t be expressive without becoming too expensive to apply at every time step?

The slide listed several choices of structured transition matrices:

$$M_t = I - \beta_t k_t k_t^\top \quad \text{DeltaNet}, \quad (9)$$

$$M_t = I - 2\beta_t k_t k_t^\top \quad \text{DeltaNet}[-1,1], \quad (10)$$

$$M_t = \prod_{p=1}^P (I - \beta_{t,p} k_{t,p} k_{t,p}^\top) \quad \text{DeltaProduct}, \quad (11)$$

$$M_t = \text{diag}(d_t) - k_t (a_t \odot k_t)^\top \quad \text{RWKV-7}. \quad (12)$$

Here the important pattern is not the exact family name, but the fact that these are low-rank, diagonal-plus-low-rank, or product-structured transitions. The speaker described these as structured enough to admit fast algorithms while being more expressive than a scalar or diagonal decay.

4.3 Why This Structure?

A student asked why the transition should have a form like $I - \beta_t k_t k_t^\top$ instead of being an arbitrary matrix. The answer was a tradeoff between expressivity and implementability.

An arbitrary dense M_t would be expressive, but the recurrence is applied at every token. A dense matrix multiply against the recurrent state would be too expensive and would require too much on-chip memory for intermediate state. At the other extreme, using the identity or a scaled identity is cheap but much less expressive. The useful middle ground is a structured matrix such as identity minus a low-rank term, a product of such terms, a diagonal-minus-low-rank matrix, or even structured permutations. These choices allow efficient multiplication and parallel algorithms while still enabling richer state transitions.

The speaker also pointed out that DeltaNet does not choose this structure arbitrarily. If one unrolls the delta-rule update, the corresponding transition has exactly this identity-minus-low-rank form. DeltaProduct can be interpreted as applying several such delta-style updates, similar in spirit to doing multiple optimization steps.

4.4 Connection to Control Theory and State Space Models

Another question asked whether these recurrences are connected to linear control theory. The answer was yes, at least at the level of overlapping model classes. The speaker distinguished two historical motivations:

- Linear attention is often motivated by starting from softmax attention and removing or replacing the softmax so that the computation can be written as a recurrence.
- State space models are often motivated from signal processing and control theory, where one writes explicit state-transition dynamics.

Although these motivations are different, many recent efficient sequence models end up with related structured recurrences. The speaker mentioned Mamba-2 as a paper that explains this overlap in detail, while also noting that the overlap is not total: some earlier state-space models and some variants such as Mamba-1 do not map as cleanly into the same framework. The practical reason for convergence is that fast parallel sequence models need transitions that are both expressive and algebraically structured enough to compute efficiently.

4.5 Associative Recall

The final topic in this part was in-context recall. Transformers have direct access to all previous tokens through attention, which makes them strong at retrieving arbitrary past information. Linear transformers are computationally attractive because they reduce compute from quadratic to linear in sequence length and can use constant-size recurrent state at inference time, but the key question is whether they can recover the same retrieval behavior.

The lecture used the multi-query associative recall task from the “Zoology” line of work as an example. A context contains key-value pairs, followed by query keys:

$$A\ 4\ B\ 3\ C\ 6\ F\ 1\ E\ 2 \longrightarrow A?\ C?\ F?\ E?\ B?. \quad (13)$$

The correct behavior is to return the associated values:

$$A \mapsto 4, \quad C \mapsto 6, \quad F \mapsto 1, \quad E \mapsto 2, \quad B \mapsto 3. \quad (14)$$

This task directly measures whether the model can retrieve information stored earlier in the context.

Historically, recurrent and linear-attention models performed poorly on such retrieval tasks. The slide showed that more recent models with data-dependent transition matrices, including DeltaNet and gated DeltaNet-style variants, do substantially better as state size grows. The intuition is that these transitions can retrieve, edit, and remove information from the state, which is exactly the operation needed for key-value recall.

4.6 Caveat

The speaker ended this segment with an important caveat. Even if recent DeltaNet variants perform well empirically on associative recall, a fixed-size recurrent state still has theoretical limitations. As the amount of information that must be retrieved from the past grows, the recurrent state may need to grow as well. Thus Delta-style linear attention improves the practical behavior of efficient sequence models, but it does not completely eliminate the fundamental tension between fixed-size state and arbitrary in-context retrieval.

5 Hybrid Models and Scaling in Practice

5.1 Hybrid Attention Models

Because of this caveat, practical systems often hybridize linear-attention and full-attention layers (Yang et al., 2024) to balance efficiency and expressiveness. These hybrid models still include occasional global softmax-attention layers, so they do not completely eliminate quadratic compute or linear KV-cache memory; however, these costs are incurred only in a subset of layers rather than throughout the entire network. Such hybrid architectures are not merely theoretical: they have already been explored in frontier models such as Qwen3-Next and Kimi Linear (Team et al., 2025).

5.2 Summary

Linear Transformers provide an efficient and expressive alternative to standard Transformers by replacing full pairwise attention with a more compact recurrent or linear state. This can greatly reduce the cost of long-context modeling, especially during inference. However, pure linear architectures still struggle with tasks that require precise associative recall, where standard attention remains very strong. For this reason, recent architectures often adopt a hybrid design: they keep some full or local attention layers to preserve strong recall and local reasoning, while using linear layers to handle longer-range information more efficiently. This makes hybrid models an appealing middle ground. At the same time, language may not be the ideal domain for evaluating these architectures, because language modeling heavily rewards exact token-level recall; other domains with smoother long-range structure may better expose the advantages of linear sequence models.

5.3 Q&A

Q1: What ratio should hybrid models use between full attention and linear/alternative attention layers? There is no stable consensus yet. The best ratio is still model-dependent and depends on what kind of attention is being mixed in. A common pattern is to use sliding-window attention for local context, because it is much cheaper than global attention, and then use DeltaNet / linear-attention-style layers to capture longer-term dependencies. Plausible ratios include something like: 1 : 3, 1 : 8, 1 : 1 meaning one attention layer may be mixed with several efficient layers, or the two types may appear in roughly equal

proportion. The high-level interpretation is that hybrid models often balance short-horizon local attention with long-horizon efficient sequence modeling.

Q2: Is this basically a long-horizon / short-horizon trade-off? Yes. Standard or sliding-window attention handles local information, while linear or sparse/efficient layers help capture longer-range relationships. The lecturer also mentions that recent models may mix compressed attention, sparse attention, local attention, and global-style mechanisms in different ways. The exact recipe varies by model, but the general idea is to combine local and long-range modeling.

Q3: Does Gemma’s attention pattern fall under this hybrid linear-attention category? Many models use separate layers, such as one layer with sliding-window attention over a fixed window like 1024 tokens, and another layer with linear attention. This is a clever and potentially interesting research idea. The main difficulty would be implementation complexity, especially managing the KV cache and linear state together.

Q4: Do purely linear or hybrid models have meaningful scaling-law properties? Mamba-2 had a scaling-law analysis and seemed to scale reasonably well. They are not sure about all newer models, but in practice, frontier labs have been trying these architectures at very large scale. The lecturer suggests that if labs such as Qwen/Kimi are investing large-scale training into these models, then they likely scale reasonably well empirically, even if not all details are public. So the answer is cautious: there is some evidence for good scaling, but the public picture is incomplete.

6 Linear Attention in Practice and System Level Optimization

6.1 SSM Cache Management Q&A

A question was asked on whether constant memory advantage of SSMs better than prefix-reuse / prefix caching. Han Guo responded that PagedAttention and other related systems handles how KV cahce is managed, however, the KV cache grows over time. In other words you can page efficiently but KV cache will eventually fill up the GPU memory and inherently fill the CPU RAM. On the flip side, Linear Attention and SSMs have static cache which makes it more efficient than KV cache. He additionally acknowledges that this advantage has not yet been fully used in open source, although he mentioned a Princeton-affiliated paper that won best paper at a recent workshop which addressed SSM cache management in practice.

6.2 Coding-Assistant Long-Context Use Case Q&A

Another question was asked regarding whether linear attention is more useful in long-context applications, such as coding assistants, for reducing inference memory usage, like the KV cache, or for improving runtime speed. Han Guo responds that linear attention offers advantages in both training and inference. In terms of training, linear attention is noticeably faster than standard attention beyond $\approx$ \$64K – \$100K. In terms of inference, the KV cache can use a lot of memory at long context lengths. SSMs can reduce this memory usage if implemented efficiently.

6.3 CODA: Rewriting Transformer Blocks as GEMM-Epilogue Programs

The lecture then transitioned to another project, which was a collaboration with researchers from Princeton, Together AI, and Meta. The motivation of the project was to take an existing trained model and make it as fast as possible while training it more efficiently, skipping the need for changing the underlying model architecture.

A 1B Llama-3-style model training on a single GPU was used as a baseline with no communication overhead. The training stack breaks down into GEMMs, attention, and the optimizer. GEMMs have been heavily optimized through lower precision formats like BF16, FP8, and MX FP4, and attention has benefited from FlashAttention, yet the remaining operations, like activation functions, cross-entropy loss, and normalization, have not been received the same level of optimization. Each of these operations is small (covering 1-2% of the training stack), but the argument is that as chips get faster, FLOP throughput improves with each hardware generation while memory bandwidth does not grow at the same rate. Hence, their relative cost in the training stack grows with every new hardware generation.

The lecture then reviews the standard transformer block (the attention sub-layer consists of a norm, QKV projection, rotary embedding, attention, and output projection into the residual). The MLP sub-layer follows a similar pattern of RSNorm, gated-up projection, SwiGLU activation, down projection, and residual accumulation.

The observation was that there exists a mathematically equivalent way of computing the same equations that merges most memory-bound operations into compute-bound kernels through a simple algebraic rewrite. The entire model can essentially be expressed using a few structurally similar kernels without the need for maintaining separate kernels for normalization, rotary embeddings, GEMMs, and attention. This not only simplifies kernel engineering for humans but also potentially makes it easier for language models to generate correct kernels as well.

7 Kernel Programming Model and Epilogue Fusion

7.1 From LLM-generated kernels to system design

LLM-based kernel authoring leads to a broader systems question: if a model can be rewritten using a small number of kernels with similar structure, then the system can expose a programming model that is easier for both humans and language models to use. The goal is not only to generate isolated CUDA snippets, but to rethink how LLM systems are built so that the high-performance parts are easier to express and maintain.

This motivates the question of what current ML compilers do well and where they still fall short. Consider a standard PyTorch function:

```
c = a @ b
if bias is not None:
    c = c + bias
if activation == "relu":
    return torch.relu(c)
if activation == "sigmoid":
    return torch.sigmoid(c)
```

`torch.compile` can trace, lower, and compile ordinary PyTorch graphs through a pipeline such as graph acquisition, graph lowering, and graph compilation. However, for an operation like matrix multiplication

followed by bias addition and an activation, `torch.compile` does not automatically solve the performance problem in the way a hand-written fused GEMM kernel would. It may generate good kernels for the elementwise activation work, but the overall computation may still not be much faster because the expensive matrix multiplication and the post-processing are not necessarily fused into the same low-level kernel.

This is why many modern LLM inference and training systems still use PyTorch mostly as a tensor orchestration layer. PyTorch is convenient for passing tensors around and connecting high-level operations, while most of the actual performance-critical computation is dispatched to bespoke GPU kernels. Frameworks such as vLLM and SGLang therefore contain a large amount of kernel engineering. This can be very fast in practice, but it is also difficult to maintain and scale because each custom kernel adds implementation complexity.

The open question is whether there is a middle ground: can we keep the maintainability and programmability of a high-level framework, while preserving the performance of carefully engineered custom kernels?

This can be viewed as an abstraction tradeoff. Torch is highly productive but may leave performance on the table; Triton, CUDA, and PTX expose progressively more control and performance potential, but at the cost of more specialized programming effort. The desired programming model would sit closer to the high-productivity side while still generating code with kernel-level efficiency.

7.2 What actually happens inside a GPU matmul

To explain where this middle ground might come from, the lecture reviewed how a simple equation like

```
output = torch.matmul(inputs, weights)
```

maps to GPU execution. In schoolbook matrix multiplication, one thinks of taking a row from matrix A , a column from matrix B , computing a dot product, and writing one entry of the output matrix C . Real GPU kernels are more complicated because performance depends heavily on the memory hierarchy and data movement.

The GPU memory system can be broken into several levels:

- **Global memory / DRAM / HBM.** This is the large off-chip memory that shows up in tools such as `nvidia-smi`. On common accelerators it may be tens of gigabytes, for example 40–80 GB. It is fast compared with CPU memory, but slow compared with on-chip GPU memory, with about 1.5 TB/s as the reference bandwidth.
- **Streaming multiprocessors (SMs).** These are the GPU cores where computation happens. Each SM has local resources such as shared memory, registers, schedulers, and tensor cores.
- **Shared memory / SRAM.** This is user-programmable on-chip memory. It is much smaller than HBM, roughly on the order of tens of MB across the chip in the A100 example, but much faster, around 19 TB/s. It is local to SMs.
- **Registers and tensor cores.** After data is staged into shared memory, smaller fragments are moved into registers and then consumed by tensor cores for matrix multiply-accumulate operations.

The practical matmul kernel therefore stages data through a tiled dataflow:

1. Copy tiles of the input and weight matrices from DRAM/HBM into shared memory.

2. Copy subtiles from shared memory into registers.
3. Dispatch the register-level fragments to tensor cores to do the matrix multiply-accumulate work.
4. Write the accumulated output back to DRAM.

This staging is the reason that a high-level expression like `torch.matmul` hides a large amount of scheduling and memory-management logic.

7.3 Epilogue: the useful work before writing back

The final writeback step motivates the **epilogue** of a GEMM kernel. The epilogue is the part near the end of the GEMM lifecycle, after the main tensor-core computation has produced accumulator tiles but before the result is written back to global memory.

The epilogue is valuable because the output data is still on chip. If we want to do post-processing such as adding a bias or applying an elementwise activation, doing it in the epilogue avoids an expensive extra round trip to DRAM. Without epilogue fusion, the system might write the raw matrix multiplication result to global memory, launch another kernel, read the result back, apply the post-processing, and write it again. That extra I/O is costly.

Another benefit is that sophisticated kernels, especially on newer architectures such as Hopper, can overlap some epilogue work with the main loop. In “ping-pong” style scheduling, tensor-core work and epilogue work can be interleaved so that the epilogue does not necessarily become a large standalone cost if it is engineered carefully.

The limitation is locality. The epilogue only sees the tile currently owned by the kernel. It does not naturally have information from other tiles. As a result, tile-local operations fit well, but operations requiring global communication across rows, columns, or multiple tiles are harder. For example, an elementwise activation is easy to fuse into an epilogue, while a full row-wise or column-wise reduction cannot simply be computed inside one tile’s epilogue without additional coordination.

7.4 Epilogue fusion example: bias and activation

A simple “Epilogue Fusion 101” example is adding a bias and then applying an elementwise activation after a matrix multiplication.

The basic sequence is:

1. Asynchronously copy the bias vector from global memory into shared memory.
2. For each accumulator subtile, copy the corresponding bias data from shared memory into registers.
3. Broadcast the bias across the accumulator tile and add it to the accumulator values.
4. Apply the elementwise activation, if needed.
5. Write the final output tile back to DRAM.

This is conceptually simple, but real epilogue construction is sophisticated enough to have its own compiler literature, such as work on epilogue visitor trees. The important point is that epilogues provide a narrow but powerful place to attach common neural-network operations without paying unnecessary memory traffic.

7.5 Toward a small set of epilogue-style primitives

The discussion then turned to how many primitives are needed for epilogue-style execution to be efficient. Rewriting a transformer forward and backward pass in this style only needs a small set of primitives, roughly five categories:

- **Elementwise and pairwise operations.** These are local transformations of accumulator values, such as residual updates, activations, RoPE rotations, and SwiGLU-style gates.
- **Rank-1 loads and stores.** These load or store vectors, such as row or column vectors, and broadcast them across a tile. Examples include RMSNorm weights, per-token scales, and optional vector-valued side outputs.
- **Rank-2 loads and stores.** These operate on matrix tiles, such as residual streams, saved activations, and intermediate values needed later by the backward pass.
- **Tile reductions.** These compute partial row-wise or column-wise reductions within an output tile, which can later be combined by a lightweight auxiliary kernel.
- **Stateful transforms.** These maintain running tile state, such as the max and sum-exp accumulators used in online log-sum-exp, softmax, and cross-entropy computations.

This primitive set is the bridge from the hardware discussion back to the programming-model discussion. Instead of maintaining many bespoke kernels by hand, the hope is to express large parts of transformer computation using a small vocabulary of tile-local epilogue operations plus a few auxiliary mechanisms for reductions and state.

8 Rewriting Transformer Modules into Epilogue Fashion

Here are examples for how CODA optimize transformer modules:

8.1 GEMM with Pairwise Actications

GEMM with Pairwise Activations refers to a fused kernel where a Matrix Multiplication is immediately followed by a non-linear operation that acts on pairs of elements within the output. This pattern is a staple of modern Transformer architectures, specifically for Gated Linear Units (GLUs) like SwiGLU or GeGLU and RoPE.

Standard activations (like ReLU or GELU) are elementwise—you apply the function to each number in the matrix independently. In contrast, pairwise activations require two values to produce one output. The most common example is the SwiGLU variant used in Llama and PaLM models:

$$\text{SwiGLU}(x, y) = \text{Swish}(x) \otimes y$$

Here, the GEMM produces a "wide" output matrix (essentially two vectors x and y for every hidden state), and the activation logic "fuses" these pairs together through multiplication.

The primary challenge of fusing these operations into a single GPU kernel stems from two main bottlenecks:

1. **Memory Layout Constraints:** Collecting pairwise inputs is highly latency-sensitive. If the corresponding elements of x and y are stored far apart in global memory, gathering them incurs significant memory-bandwidth overhead.
2. **Dynamic Matrix Shapes:** The structural changes in matrix dimensions (e.g., halving the channel dimension post-activation) complicate clean vectorization and thread-block tiling strategy.

The NVIDIA Hopper architecture (H100) introduces a paradigm shift in handling GEMM with pairwise activations, as a native hardware-software co-design. Hopper resolves the pairwise adjacency problem via the Tensor Memory Accelerator (TMA) and Epilogue Visitor Trees (EVTs). The TMA asynchronously swizzles layouts directly into Shared Memory so that **pairs are physically adjacent before processing**, allowing the SM to execute the pairwise activation seamlessly as a pipelined, register-efficient epilogue before writing the final halved matrix back to VRAM.

8.2 GEMM-Residual-RMSNorm-GEMM Pattern

GEMM-Residual-RMSNorm-GEMM Pattern is one of the most common computation pattern found in modern deep learning architectures (like Transformers). It usually consists of the following operations:

$$y = \text{RMSNorm}(xW_0 + z, \gamma)W_1 = (r(xW_0 + z) \odot \gamma) W_1$$

Hardware typically handles this sequence when it is not optimized (un-fused):

- First GEMM : Takes the inputs and weights to compute the matrix multiplication. This operation is compute-bound (limited by the GPU’s processing power).
- The Memory-Bound Bridge : The outputs of the first GEMM are written to global memory, read back to perform the RMSNorm and residual addition, and written back to memory again as "Outputs". These intermediate steps are **memory-bound kernels**. The GPU spends more time waiting to read/write data from its memory (VRAM) than it does actually computing.
- Second GEMM: Takes the final normalized outputs and a new set of weights to compute the next matrix multiplication.

A crucial mathematical optimization for this pattern is to utilize an algebraic trick that enables kernel fusion to bypass a major GPU memory bottleneck:

$$y = (r(xW_0 + z) \odot \gamma) W_1 = r((xW_0 + z) \odot \gamma) W_1$$

The Right Side of the equation above (Optimized Flow) shows that the scalar or row-wise RMSNorm scaling factor r is pulled outside of the inner expression, commuting past the brackets. This is because the RMSNorm scale is shared across rows, so it commutes with the second GEMM

Using this trick, CODA reorganizes how the data flows through the hardware by splitting the normalization process into a parallel path:

- Fusing the GEMMs: Because the RMSNorm scale commutes, the full-sized matrix output from the first GEMM can bypass the normalization step and feed directly into the second GEMM.
- The Reduced Path: Instead of passing the entire full-sized matrix through a normalization kernel, a highly compact intermediate state is passed to a much lighter Reduce kernel.

- A Slimmer Reduction: The Reduce kernel only needs to calculate the row-wise root-mean-square scaling factor. This means the memory-bound footprint is dramatically shrunk down to a thin vector.
- Final Fusion: This tiny scaling vector is then applied directly during the second GEMM computation.

This reduces the memory bound kernel IO by order(s) of magnitude.

8.3 GEMM with Cross Entropy Loss

In large models, this step is notoriously memory-heavy because multiplying the model’s hidden states by a massive vocabulary matrix generates an enormous logits tensor before reducing it down to a scalar loss.

The cross entropy formula is split into two structural requirements pointed out by the arrows:

- Index GEMM (The Positive Term): You only need to compute the logit for the actual ground-truth token.
- Reduction (The Negative Log-Sum-Exp Term): To calculate the denominator of the softmax, you must sum over the exponential of the logits for all tokens in the entire vocabulary.

Two optimization strategies were introduced for the forward pass:

- Indexed Matmul: Instead of multiplying the full hidden state embedding matrix by the entire transposed vocabulary weight matrix (which would create a massive tensor of size $\text{Batch} \times \text{Sequence} \times \text{Vocab}$), it performs an indexed load. It pulls only the specific row vectors corresponding to the target ground-truth tokens. It then performs a quick dot product with the hidden state. This yields the target logit directly (the green bar at the bottom) without ever computing or storing the logits of the thousands of incorrect vocabulary words.
- Linear-Log-Sum-Exp (Forward Pass): To get the log-sum-exp (LSE_n) denominator without running out of VRAM, the computation is tiled and streamed. The vocabulary matrix (dimension $|V|$) and the embedding hidden states (dimension N) are processed via Block Matmul. As soon as a block matrix multiplication is computed in fast local memory, it is immediately fed into a Block LSE operator (LSE_{nv}) ($\log(\exp(\text{LSE}_n) + \exp(\text{LSE}_{nv}))$). This continuously updates the running log-sum-exp scalar vector (LSE_n) across the tiles. The massive intermediate logit matrix is consumed on-the-fly and immediately discarded, reducing memory overhead from $O(\text{Sequence} \times \text{Vocab})$ down to a fraction of that size.

Similarly to the forward pass, the backward pass is also rewritten into several GEMM-Epilogue block.

9 CODA Block Benchmarks and Q&A

Block-level benchmarks compare CODA (LLM-generated and human-written kernels) against Liger and FlashInfer, using cuBLAS + PyTorch as the baseline, across hidden dimensions $d \in \{2048, 4096, 8192\}$. CODA achieves speedups on both the forward and backward passes, most noticeably at smaller d where the workload is memory-bound. Liger falls below baseline on the backward pass, showing that naive fusion without careful backward-pass design can hurt performance. Han Guo noted that per-device workload in large distributed training is effectively small- d , so these gains are practically meaningful at scale.

On the question of AI-automated kernel engineering: language models are already good at algorithm-level reasoning but struggle with low-level primitives such as TMA instructions and hand-written PTX, due to

scarcity of open-source optimized kernels. The practical approach is a division of labor—humans provide efficient, composable primitives; language models compose them into kernels.

On rematerialization in the backward pass: CODA’s backward kernel recomputes activations (e.g., SwiGLU outputs) within a single tile and fuses the gradient computation as an epilogue, keeping all intermediates register-local and avoiding global memory round-trips. The fusion rules for the backward pass must be derived manually, as patterns like SwiGLU introduce asymmetry relative to the forward pass. Han Guo observed that the backward pass is almost always the harder and more expensive part to engineer—in one case, a forward kernel took one week in Triton while the backward took a month—and encouraged treating backward-pass efficiency as a first-class concern when designing new architectures.

On MoE extensions: the current CODA framework is specialized for dense Llama-3 style architectures and it remains an open question how to extend the fusion approach to MoE routing or gated attention.

References

- Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 5156–5165. PMLR, 2020. URL <https://arxiv.org/abs/2006.16236>.
- Kimi Team, Yu Zhang, Zongyu Lin, Xingcheng Yao, Jiayi Hu, Fanqing Meng, Chengyin Liu, Xin Men, Songlin Yang, Zhiyuan Li, et al. Kimi linear: An expressive, efficient attention architecture. *arXiv preprint arXiv:2510.26692*, 2025.
- Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, and Yoon Kim. Parallelizing linear transformers with the delta rule over sequence length. In *Proceedings of the 38th International Conference on Neural Information Processing Systems*, NIPS ’24, Red Hook, NY, USA, 2024. Curran Associates Inc. ISBN 9798331314385.